

1. Базовые понятия RL

Обучение с подкреплением (RL) описывает взаимодействие агента со средой. Математически этот процесс записывается как Марковский процесс принятия решений, состоящий из пяти простых элементов $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$:

- $\mathcal{S}$ (States) — все возможные ситуации (состояния), в которых может оказаться агент.
- $\mathcal{A}$ (Actions) — все действия, которые агент может совершить.
- $P(s' | s, a)$ — физика среды. Вероятность того, что при действии a в состоянии s мы попадем в состояние s' .
- $R(s, a)$ — награда (число), которую среда дает агенту за совершенное действие.
- $\gamma \in [0, 1]$ — коэффициент дисконтирования. Он показывает, насколько нам важна будущая выгода по сравнению с мгновенной.

В процессе игры агент генерирует последовательность — траекторию τ :

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots) \quad (1)$$

Цель агента — максимизировать общую сумму наград с учетом дисконтирования (эту величину называют отдачей G_t):

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2)$$

Если $\gamma = 0$, агент думает только о сиюминутной награде. Чем ближе γ к 1, тем более дальновидным становится агент.

2. Метод кросс-энтропии

Метод кросс-энтропии — это простой «эволюционный» алгоритм. Его суть: сыграть много игр, отобрать самые лучшие (элитные) и обучить агента повторять действия из этих лучших игр.

2.1. Табличный случай

Если состояний и действий мало, мы можем записать политику $\pi(a | s)$ как таблицу, где на пересечении строки состояния и столбца действия стоит вероятность выбора этого действия. Алгоритм:

1. Генерируем N игр по текущей таблице вероятностей.

2. Сортируем игры по их суммарной награде и берем топ-30% лучших игр (назовем их «элитными»).
3. Пересчитываем вероятности в таблице: новое значение — это просто частота, с которой действие a совершалось в состоянии s внутри элитных игр:

$$\pi(a | s) = \frac{\text{Кол-во переходов } (s \rightarrow a) \text{ в элитных играх}}{\text{Общее кол-во визитов в состояние } s \text{ в элитных играх}} \quad (3)$$

2.2. Аппроксимация нейросетью

Если состояний слишком много (например, экран игры), таблицу хранить нельзя. Вместо нее мы используем нейросеть $\pi_\theta(a | s)$ с весами θ , которая по состоянию предсказывает вероятности действий.

После того как мы отобрали элитные игры, мы обучаем нейросеть методами классического обучения с учителем. В качестве таргетов выступают действия из элитных эпизодов. Мы минимизируем функцию потерь кросс-энтропии:

$$\mathcal{L}(\theta) = - \sum_{(s,a) \in \text{elite}} \log \pi_\theta(a | s) \quad (4)$$

3. $V(s)$, $Q(s, a)$ и уравнения Беллмана

Функции ценности помогают агенту понять, насколько перспективна текущая ситуация.

Функция ценности состояния $V(s)$ (Value) — оценивает, насколько хорошо находиться в состоянии s , если дальше мы будем играть по нашей текущей стратегии:

$$V(s) = \mathbb{E}[\text{Будущая награда } G_t | \text{Старт из } s] \quad (5)$$

Функция ценности действия $Q(s, a)$ (Quality) — оценивает, насколько хорошо в состоянии s выбрать действие a , а уже затем продолжить играть как обычно:

$$Q(s, a) = \mathbb{E}[\text{Будущая награда } G_t | \text{Старт из } s, \text{ делаем действие } a] \quad (6)$$

Связь между ними проста: $V(s)$ — это среднее значение $Q(s, a)$ по всем действиям.

Уравнения Беллмана связывают ценность текущего шага со следующими шагами. Их суть: «ценность сейчас равна полученной награде плюс ценность того места, куда мы попадем дальше».

В простом виде для ожидаемой ценности:

$$V(s) = \text{Награда } R + \gamma \cdot \text{Средняя ценность следующего состояния } V(s') \quad (7)$$

Для оптимальных (наилучших возможных) функций уравнения Беллмана используют максимум:

$$V^*(s) = \max_a Q^*(s, a) \quad (8)$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \quad (9)$$

4. Метод Policy iteration

Итерация по политике — это классический способ найти идеальную стратегию, если мы заранее знаем, как устроена среда (знаем вероятности переходов $P(s' | s, a)$). Метод бесконечно повторяет два шага:

1. Policy Evaluation: Берем текущую стратегию агента π и считаем для нее точные значения ценностей $V(s)$ для всех состояний. Для этого мы многократно обновляем значения по формуле:

$$V(s) \leftarrow \text{Награда} + \gamma \cdot \text{Среднее } V(s') \quad (10)$$

Повторяем это, пока цифры в таблице V не перестанут меняться.

2. Policy Improvement: Обновляем нашу стратегию. Теперь в каждом состоянии s агент будет выбирать то действие, которое ведет к максимальной ценности (то есть ведет себя «жадно» относительно посчитанной V):

$$\pi(s) \leftarrow \arg \max_a Q(s, a) \quad (11)$$

Этот цикл гарантированно находит самую оптимальную стратегию.

5. Q-learning и SARSA

Когда мы не знаем законов среды заранее, мы учимся на собственном опыте по методу временных различий. Агент делает шаг, получает награду и обновляет свое представление о ценности $Q(s, a)$.

SARSA (On-policy): Этот алгоритм обучается на действиях, которые агент реально совершает. Агент находится в S , делает действие A , получает награду R , переходит в S' и выбирает там следующее действие A' по своей текущей стратегии. Формула обновления:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)] \quad (12)$$

где α — learning rate.

Q-learning: Этот алгоритм оценивает будущие шаги оптимистично. Ему все равно, какое действие агент реально выберет на следующем шаге. Для обновления он предпола-

гает, что выберет наилучшее из возможных действий:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right] \quad (13)$$

Главное отличие: SARSA более осторожна (учитывает свои будущие случайные метания/исследования при обучении). Q-learning более агрессивен и учит идеальную стратегию, предполагая, что в будущем ошибок совершаться не будет.

6. Exploration vs Exploitation

Это фундаментальная дилемма: использовать то, что уже хорошо работает (Exploitation), или пробовать новое и неизвестное (Exploration).

1. ϵ -greedy (Эпсилон-жадная стратегия): С большой вероятностью $1 - \epsilon$ агент выбирает лучшее известное действие, а с маленькой вероятностью ϵ — абсолютно случайное действие. Обычно со временем ϵ уменьшают.

2. Boltzmann exploration (Softmax): Вероятность выбора действия зависит от его ценности Q . Чем выше $Q(s, a)$, тем чаще мы его выбираем, но шанс выбрать плохие действия все равно остается. Регулируется температурой T :

$$P(a) = \frac{\exp(Q(s, a)/T)}{\sum \exp(Q(s, a')/T)} \quad (14)$$

При высоком T действия выбираются почти случайно, при низком — почти всегда берется самое лучшее.

3. UCB (Upper Confidence Bound): Принцип «оптимизма в условиях неопределенности». Мы выбираем действие по формуле:

$$A = \arg \max_a [Q(a) + \text{Бонус за редкость}(a)] \quad (15)$$

Если мы редко выбирали какое-то действие, у него будет огромный «бонус за редкость», что заставит агента его проверить.

7. DQN (Deep Q-Network)

DQN — это Q-learning, в котором вместо таблицы используется глубокая нейросеть для предсказания $Q(s, a)$. Обучение идет через минимизацию квадратичной ошибки:

$$\mathcal{L}(\theta) = (\text{Цель (Target)} - Q(s, a; \theta))^2 \quad (16)$$

где $\text{Target} = r + \gamma \max_{a'} Q(s', a'; \theta^-)$.

Обучать нейросеть стандартным способом в RL сложно из-за двух проблем:

1. Зависимость данных: В игре кадры идут друг за другом и очень похожи. Нейросеть

забывает старое и переобучается под текущий момент.

2. Движущаяся цель: Мы обновляем веса сети θ , но от этих же весов зависит и наша цель (Target). Это похоже на попытку стрелять по мишени, которая привязана к дулу вашего пистолета.

DQN решает эти проблемы с помощью Experience Replay (буфер опыта) и Target Network (целевая сеть).

8. Experience replay

Чтобы данные не были похожи друг на друга, DQN использует буфер памяти $\mathcal{D}$.

1. Во время игры все переходы (s, a, r, s') записываются в этот буфер.
2. Вместо того чтобы учиться на последнем шаге, алгоритм берет из буфера случайную батч старых воспоминаний.

Случайный выбор данных уничтожает временную связь между кадрами (делает данные независимыми и одинаково распределенными — i.i.d.). Это позволяет использовать стабильный градиентный спуск, как в обычном Deep Learning.

9. Target network

Чтобы решить проблему движущейся цели, в DQN создают копию нейросети.

- Основная сеть (θ): обучается на каждом шаге, выбирает действия.
- Целевая сеть (θ^-): используется только для расчета правильных ответов (Target). Ее веса заморожены и обновляются редко.

Обновление целевой сети бывает двух видов:

1. Hard update: Раз в C шагов полностью копируем веса: $\theta^- \leftarrow \theta$.
2. Soft update: На каждом шаге плавно сдвигаем целевую сеть в сторону основной с маленьким шагом $\tau \ll 1$:

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^- \quad (17)$$

10. Double DQN

Проблема переоценки в Q-learning: Поскольку в формуле цели используется оператор максимума $\max_a Q(s, a)$, алгоритм склонен сильно завышать ценность действий. Если оценки ценностей содержат даже случайный шум, максимум всегда выберет завышенную

из-за шума оценку. В итоге агент начинает верить, что его действия намного лучше, чем они есть на самом деле.

Решение Double DQN: Мы разделяем выбор лучшего действия и оценку его ценности между двумя сетями:

1. С помощью основной сети θ мы находим, какое действие сейчас кажется наилучшим:
 $a^* = \arg \max_a Q(s', a; \theta)$.
2. Но ценность этого действия оцениваем с помощью целевой сети θ^- :

$$\text{Target} = r + \gamma Q(s', a^*; \theta^-) \quad (18)$$

Это убирает систематическое завышение оценок.

11. Policy gradient

Вместо того чтобы оценивать ценность действий (Q -функцию), методы градиента политики напрямую обучают нейросеть $\pi_\theta(a | s)$ выдавать вероятности действий. Мы хотим настроить веса θ так, чтобы увеличить вероятность хороших действий и уменьшить вероятность плохих.

Главная формула:

$$\nabla_\theta J(\theta) = \mathbb{E} [\nabla_\theta \log \pi_\theta(a | s) \cdot G] \quad (19)$$

Простыми словами: $\nabla_\theta \log \pi_\theta(a | s)$ показывает, в какую сторону нужно изменить веса, чтобы повысить вероятность выбора действия a в состоянии s . А величина отдачи G работает как множитель: если мы получили большую награду ($G > 0$), мы сильно двигаем веса в эту сторону. Если награда маленькая или отрицательная, мы уменьшаем вероятность этого действия.

12. Алгоритм REINFORCE

REINFORCE — это самый простой алгоритм градиента политики, работающий по методу Монте-Карло:

1. Агент играет игру от начала до самого конца.
2. Для каждого шага t рассчитывается реальная полученная отдача G_t .
3. Веса сети обновляются:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \quad (20)$$

Baseline: Поскольку игры бывают очень случайными, REINFORCE страдает от высокой дисперсии. Чтобы это исправить, из полученной отдачи вычитают некоторый

baseline $b(s)$ — например, среднюю награду в этом состоянии:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)(G_t - b(s_t)) \quad (21)$$

Если сыграли лучше, чем обычно ($G_t - b > 0$) — действие поощряется. Если хуже среднего — наказывается. Математически доказано, что вычитание baseline не ломает сходимость алгоритма, но сильно ускоряет обучение.

13. Архитектура Actor-Critic

Актор-Критик объединяет два подхода: градиент политики и оценку ценности. Архитектура состоит из двух нейросетей:

1. Актор (Actor) — Политика $\pi_{\theta}(a | s)$: Выбирает действия.
2. Критик (Critic) — Оценка ценности $V_w(s)$: Оценивает, насколько хороша ситуация, в которой оказался Актор.

Как они работают вместе на каждом шаге:

1. Актор делает шаг, среда возвращает награду r и новое состояние s' .
2. Критик оценивает ошибку прогноза (TD-ошибку δ):

$$\delta = r + \gamma V_w(s') - V_w(s) \quad (22)$$

Эта ошибка показывает, стало ли агенту жить лучше ($\delta > 0$) или хуже ($\delta < 0$), чем ожидал Критик.

3. Актор обновляет свои веса, используя δ : если шаг превзошел ожидания, вероятность действия увеличивается.
4. Критик подстраивает свои веса w , чтобы точнее предсказывать ценность состояния в будущем.

14. RNN для генерации последовательностей

Генерация текста — это предсказание следующего токена на основе всех предыдущих:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \quad (23)$$

Рекуррентные нейронные сети (RNN) сжимают всю историю генерации в вектор скрытого состояния h_t , который обновляется на каждом шаге:

$$h_t = \text{RNN-Cell}(h_{t-1}, x_t) \quad (24)$$

На основе h_t сеть через Softmax выдает вероятности для следующего слова. Обучение идет стандартной минимизацией кросс-энтропии.

Режим Обучение с учителем: При генерации модель может ошибиться на 1-м шаге, подать свою ошибку на вход на 2-м шаге, и дальше пойдет бессвязный бред. Чтобы этого избежать, при обучении на вход модели всегда подают истинные слова из текста, а не те, которые она сама случайно предсказала на предыдущем шаге. Это делает обучение стабильным и быстрым.

15. Дообучение генеративных моделей с помощью RL

Иногда нам нужно дообучить языковую модель так, чтобы ее тексты были не просто грамматически правильными, а добрыми, полезными или соответствовали заданной метрике (например, BLEU или оценкам людей). Обычным обучением с учителем этого не сделать, так как «доброта» или метрика BLEU не дифференцируемы.

Мы представляем генерацию текста как задачу RL:

- Состояние: уже написанная часть предложения.
- Действие: выбор следующего слова.
- Политика: наша языковая модель π_θ .
- Награда: оценка от человека или специальной Reward-модели за весь написанный текст.

Проблема «слома» языка: Если просто натравливать RL на модель, она быстро научится «хакать» награду — например, бесконечно повторять кодовые вежливые слова, теряя человеческий смысл.

Чтобы этого не произошло, к награде добавляют штраф за слишком сильное отклонение от исходной (хорошей предобученной) модели π_{ref} через KL-дивергенцию:

$$\text{Итоговая награда} = \text{Оценка} - \beta \cdot D_{KL}(\pi_\theta \parallel \pi_{ref}) \quad (25)$$

Штраф KL не дает новой модели уйти слишком далеко от правил русского языка, гарантируя, что генерируемый текст останется связным и естественным.